Experiment 15: Introduction to Docker and Basic Commands

Install Docker (Basic Steps)

Windows

1. Download Docker Desktop from:
2. `https://www.docker.com/products/docker-desktop`
3. Run the installer.
4. Restart the system.
5. Open Command Prompt.

Verify installation

```
docker --version
```

Expected output:

```
Docker version XX.X.X
```

Basic Docker Commands

Check Docker version

```
docker --version
```

List running containers

```
docker ps
```

List all containers

```
docker ps -a
```

List images

```
docker images
```

Task 1 — Run First Container

Run a sample container:

```
docker run hello-world
```

What happens:

1. Docker checks for image locally.
2. If not found, it downloads it.
3. Creates a container.
4. Runs the container.
5. Displays a success message.

Task 2 — Run a Web Server Container

Pull Nginx image:

```
docker pull nginx
```

Run container:

```
docker run -d -p 8080:80 nginx
```

Open browser:

```
http://localhost:8080
```

You will see the Nginx welcome page.

Task 3 — Manage Containers

List containers:

```
docker ps
```

Stop container:

```
docker stop <container_id>
```

Remove container:

```
docker rm <container_id>
```

Experiment 16: Creating Docker Images using Dockerfile

Objective

- Understand what a Docker image is.
- Learn how to create images using a **Dockerfile**.
- Build and run a custom Docker image.

What is a Docker Image?

- A Docker image is a **read-only template**.
- It contains:
  - Application code
  - Runtime
  - Libraries
  - Dependencies
- Used to create containers.

Flow:

```
Dockerfile → Image → Container
```

What is a Dockerfile?

- A **text file** containing instructions.
- Defines how a Docker image is built.
- Each instruction creates a layer in the image.

Common Dockerfile Instructions

Instruction	Purpose
FROM	Base image
WORKDIR	Set working directory
COPY	Copy files into image
RUN	Execute commands
EXPOSE	Open port
CMD	Default command to run

Task 1 — Create a Simple Application

Create a folder:

```
mkdir docker-app
cd docker-app
```

Create a file named:

```
         1) sample.py
```

Add the following code:

```
print("Hello from Docker container!")

2)app.py (webserver code)
```

Add the following code:

```
from http.server import SimpleHTTPRequestHandler, HTTPServer

PORT = 5020

class MyHandler(SimpleHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(b"<h1>Hello from Docker Python Container</h1>")

server = HTTPServer(("0.0.0.0", PORT), MyHandler)
print(f"Server running on port {PORT}...")
server.serve_forever()
```

Task 2 — Write the Dockerfile

Create a file named:

```
   1) Dockerfile for sample.py
```

Add:

```
FROM python:3.9-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]

   1) Dockerfile for app.py
      FROM python:3.9-slim

      WORKDIR /app

      COPY app.py .

      EXPOSE 5020

      CMD ["python", "app.py"]
```

Task 3 — Build the Docker Image

Run in the project folder:

```
docker build -t my-python-app .
```

Explanation:

- `-t` → tag (name of image)
- `.` → current directory (contains Dockerfile)

Task 4 — Check Created Image

```
docker images
```

You should see:

```
my-python-app
```

Task 5 — Run the Container

```
docker run my-python-app
```

Expected output of sample.py:

```
Hello from Docker container!
```

docker run -p 5000:5000 my-python-app1

Expected output of app.py:

```
Hello from Docker Python Container in web browser
```